



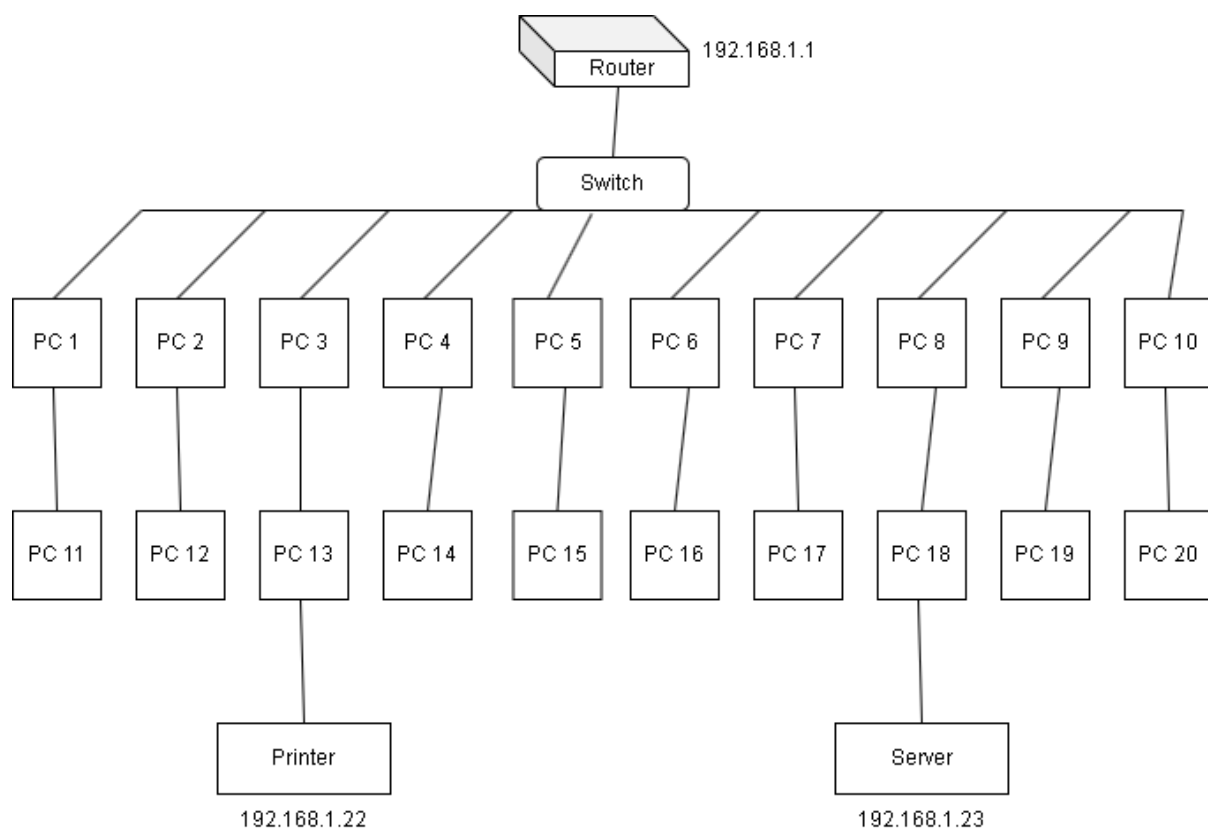
Title: Networking and Security Operations with SIEM, Forensics, and Traffic Analysis

Introduction

In modern networks, maintaining secure and efficient communication is essential due to increasing cyber threats. This task focuses on combining networking fundamentals with security operations, including subnetting, traffic analysis, and protocol troubleshooting.

It also involves setting up a SIEM system using the ELK Stack and performing network forensics to investigate a simulated security incident. Additionally, threat hunting techniques are used to identify potential indicators of compromise (IOCs), highlighting the importance of monitoring and analyzing network and log data for effective security.

Subnet Design



- Network: 192.168.1.0/27
- I take CIDR /27
- Subnet Mask is 255.255.255.224
- So: $2^{(32-27)} = 32$
 - Which means it gives us 32 total IPs.



- And 30 usable IPs and requirement was 20 so it fits.
 - Network address is 192.168.1.0
 - Usable range is 192.168.1.1 - 192.168.1.30
 - Since /27 is smallest it supports 20 devices and more and it will be minimal wastage of IPs as well.
-

Analyzing Network Traffic Patterns

Key Protocol Identifies

TCP (Transmission Control Protocol) - Reliable communication (HTTP,HTTPS)

- I observed 3-way handshake (SYN to SYN-ACK to ACK)
- There was also retransmissions written on some streams which means possible network delay.
- Wireshark Filter used was "tcp"

UDP (User Datagram Protocol) - Fast communication (DNS, streaming)

- There was no handshake in it since i didn't do the udp scan because the firewall was blocking the scan in the vulnerable machine so i had to use the -f command to send the packets in fragments for the scan.
- And since there were mostly DNS queries as well in it.
- Wireshark Filter was "udp"

DNS (Domain Name System) - Resolving domain names to IP addresses

- This protocol was very frequent to public DNS servers.
- Some bursts and spikes in DNS traffic was observed.

Highest Bandwidth Users

- Source IP was 192.168.0.0 and the Destination IP was 192.168.0.0.
- Sending Packets was 67535 to 70000+

Unusual Patterns Detected

- Sudden increase in DNS requests since i was scanning a Web Application.
 - TCP Retransmission Detected that might be because of Packet Loss or Network Congestion.
 - Suspicious IP Communication was involved which was unknowns IPs and Repeated connections.
-

Troubleshooting Network Protocol Issues



If ping to 8.8.8.8 but ping to google.com is not successful that means there is a DNS configuration problem. Which can also mean that the domain is not correct that we are pinging to or it means the network is having a problem with resolving DNS. To solve this:

- We can verify if IP connectivity is working or not.
- Check the DNS configuration if it is wrong.
- Observe DNS traffic in Wireshark to analyze if there are no replies.
- Identify the root cause as well and it also means to check if there is a wrong DNS server or not.
- We can check using nslookup as well.

Setting up ELK stack for SIEM

Installing ELK stack

1. To setup ELK stack on Linux we have to install Elasticsearch, Logstash, and Kibana and configure them.
2. Run `sudo apt install elasticsearch` to install Elasticsearch.
 - a. First we have to install prerequisites as well.
 - b. Run `sudo apt install -y apt-transport-https wget gnupg`
 - c. Then `wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg`
 - d. And `echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/8.x/apt stable main" | sudo tee /etc/apt/sources.list.d/elasticsearch-8.x.list`
 - e. Then you have to run the update command again.
3. Run `sudo apt install logstash` to install Logstash.
4. Run `sudo apt install kibana` to install Kibana.

Configuring ELK stack

1. First we will configure Elasticsearch so in terminal type `nano /etc/elasticsearch/elasticsearch.yml` and we have to change some things.

```
#
network.host: 0.0.0.0
#
# By default Elasticsearch listens for HTTP traffic on the first free port it
# finds starting at 9200. Set a specific HTTP port here:
#
http.port: 9200
#
# For more information, consult the network module documentation.
#
```

2. As shown in the image we have to uncomment `network.host` and type `0.0.0.0` with it.
3. Next, we must uncomment `http.port: 9200`
4. We must uncomment `cluster.name` and `node.name` at the top as well.
5. Save and exit the file.



6. Now start the service by using “**systemctl start elasticsearch**”.
7. Configuring Kibana now. First go to “nano /etc/kibana/kibana.yml”
8. Uncomment “server.port: 5601”
9. Uncomment “server.host: “localhost”” and change the localhost to 0.0.0.0 so it should look like “server.host: “0.0.0.0””.
10. Now Uncomment “server.name: “demo-name”” change the name if you want.
11. Next Uncomment “elasticsearch.hosts: [<http://localhost:9200>]”

```
# Kibana is served by a back end server. This setting specifies the port to use.
server.port: 5601

# Specifies the address to which the Kibana server will bind. IP addresses and host names are both valid values.
# The default is 'localhost', which usually means remote machines will not be able to connect.
# To allow connections from remote users, set this parameter to a non-loopback address.
server.host: "0.0.0.0"

# Enables you to specify a path to mount Kibana at if you are running behind a proxy.
# Use the 'server.rewriteBasePath' setting to tell Kibana if it should remove the basePath
# from requests it receives, and to prevent a deprecation warning at startup.
# This setting cannot end in a slash.
#server.basePath: ""

# Specifies whether Kibana should rewrite requests that are prefixed with
# 'server.basePath' or require that they are rewritten by your reverse proxy.
# Defaults to 'false'.
#server.rewriteBasePath: false

# Specifies the public URL at which Kibana is available for end users. If
# 'server.basePath' is configured this URL should end with the same basePath.
#server.publicBaseUrl: ""

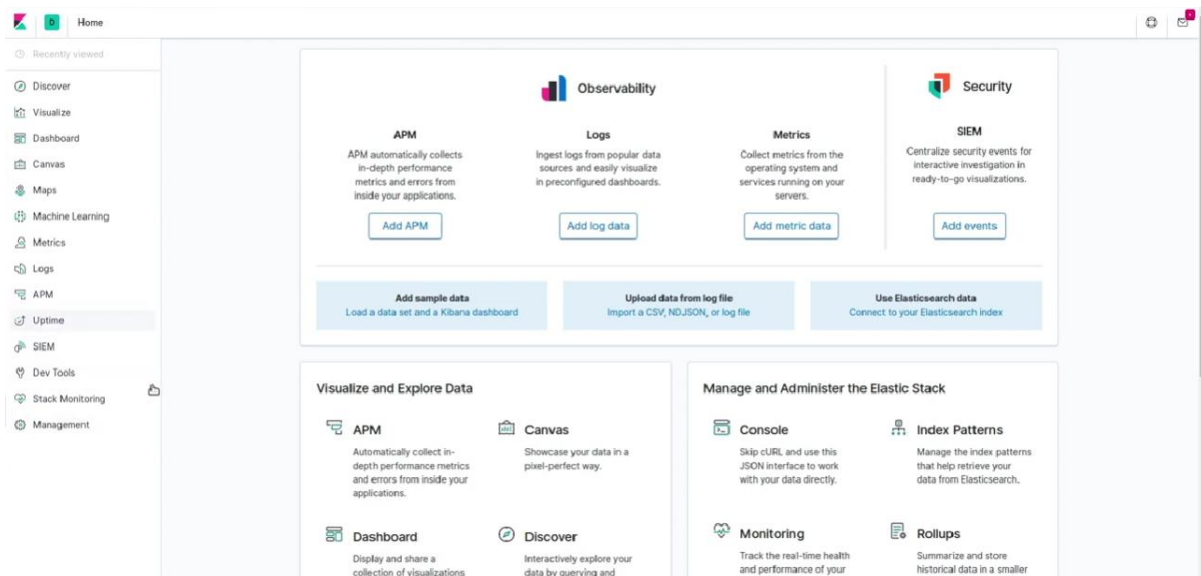
# The maximum payload size in bytes for incoming server requests.
#server.maxPayload: 1048576

# The Kibana server's name. This is used for display purposes.
server.name: "raeven"

# ===== System: Kibana Server (Optional) =====
# Enables SSL and paths to the PEM-format SSL certificate and SSL key files, respectively.
# These settings enable SSL for outgoing requests from the Kibana server to the browser.
#server.ssl.enabled: false
#server.ssl.certificate: /path/to/your/server.crt
#server.ssl.key: /path/to/your/server.key

# ===== System: Elasticsearch =====
# The URLs of the Elasticsearch instances to use for all your queries.
elasticsearch.hosts: ["http://localhost:9200"]
```

12. It should look something as the image above and we can leave rest of the values as it is right now so just save the file and exit it.
13. We can start kibana as well now with “**systemctl start kibana**”.
14. Then go to the browser and in the URL section type <http://localhost:5601> and it should start kibana.



15. We should see something like this.

Conclusion and Key Learnings

This report helps us understand how to monitor using ELK stack for multiple login attempts and using Wireshark to see what the packet is responding to the request. We also learned on how to install and setup Elasticsearch with Kibana and we now know that if Elasticsearch is not working then Kibana won't work as well. We fixed Network Protocol issues and even analyzed for top talkers and unusual patterns using Wireshark.